Binär & Mehr

Ein C/C++-Kurs für junge Entdecker

11111101

11111101

11111101

Dr. rer. nat. Jürgen Sawinski

Copyright © 2026 Jürgen “George” Sawinski

VERÖFFENTLICHT VOM AUTOR

https://github.com/jsawinski/twin-c-course

Licensed under the MIT License

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Umschlag: Gestaltet von DeepSeek AI auf der Grundlage des Binärmotivs 11111101 – eine Hommage an die koreanische Band Idiotape und ein wiederkehrendes visuelles Leitmotiv dieses Buches.

Dieser Kurs ist gedacht für Kinder ab Ende der Grundschule (ca. 5. Klasse), die Begeisterung für Mathematik und Computer mitbringen und den Drang haben, programmieren zu lernen.

Der erste Teil des Kurses beschäftigt sich damit, alle grundlegenden Kenntnisse zu erwerben, um eine Konsolen-basierte Regensimulation in C zu programmieren, was auch mathematische und physikalische Grundlagen beinhaltet.

Der zweite Teil des Kurses wird das Ziel haben, das Spiel Tetris® in C++ umzusetzen, wobei das Hauptaugenmerk dabei auf die Grundlagen Objekt-orientierter Programmierung gelegt wird.

Inhaltsverzeichnis

I Die Programmiersprache C **1**

1 Ein paar Grundlagen **3**
1.1 Das Binärsystem 3
1.2 Logikgatter — die Bausteine des Prozessors 5
1.3 Bitweise Operationen in C 7

2 Erste Schritte in C **9**
2.1 Programm: Hallo Welt! 9
2.2 Programm: Zahlen 11
2.3 Programm: Fibonacci 13
2.4 Programm: Reihen 15
2.5 Programm: Lauflicht 18
2.6 Programm: Lauflicht - Vertikal 21
2.7 Programm: Ein Würfel 25
2.8 Programm: Das Spielfeld 30
2.9 Warnungen des Compilers 35

3 Etwas Physik **37**
3.1 Bewegung 37

4 Regensimulation in C **39**
4.1 Warnungen des Compilers 39

II Die Programmiersprache C++ **41**

III Anhang **43**

A Software **45**

B Leitfaden **47**
B.1 Tastenkombinationen 47
B.1.1 Sonderzeichen 48

B.1.2 Referenz 48
B.2 C 49
B.2.1 Grundlagen 49
B.2.2 Datentypen 49
B.2.3 Strukturen 51
B.2.4 Anweisungsblock 51
B.2.5 Kontrollstrukturen 52
B.2.6 Schleifen 54
B.2.7 Funktionen 55
B.2.8 Namen 56
B.2.9 Referenz 57

C Lösungen zu den Übungen **63**

Index **69**

Vorwort

Dieser Kurs ist aus der gemeinsamen Arbeit des Autors mit seinen Söhnen entstanden. Die Idee, einen C/C++-Kurs für Kinder zu schreiben, kam von den Jungs selbst, die unbedingt programmieren lernen wollten um ihre eigenen Spiele zu entwickeln zu können.

Die Mitschrift ist in erster Linie für Kinder gedacht, die den Kurs mit einer Begleitperson zusammen durchlaufen und dient als Leitfaden sowie als Nachschlagewerk für die behandelten Themen.

Der Autor hat die Inhalte erarbeitet, erklärt und in LaTeX festgehalten. Bei der Strukturierung des Materials und der Erstellung einiger LaTeX-Makros wurde er durch KI-gestützte Werkzeuge (**DeepSeek** und **GitHub Copilot**) unterstützt.

DeepSeek lieferte Vorschläge zur Gliederung, zur didaktischen Aufbereitung und zu technischen Umsetzungen in LaTeX. Github Copilot wurde beim Schreiben von LaTeX-Code und bei der Erstellung von C/C++-Codebeispielen eingesetzt. Alle Vorschläge wurden vom Autor geprüft, angepasst und auf die Bedürfnisse seiner Kinder zugeschnitten.

– Jürgen "George" Sawinski, 23. Oktober 2026

Lizenz und Haftungsausschluss

Alle Inhalte dieses Kurses wurden vom Autor selbst erstellt und sind, soweit nicht anders gekennzeichnet, frei von urheberrechtlich geschützten Materialien Dritter. Die Beispiele und Übungen sind so gestaltet, dass sie die Lernziele unterstützen und den Kindern helfen, die Konzepte zu verstehen und anzuwenden.

Dieses Werk wird unter der **MIT-Lizenz** veröffentlicht. Das bedeutet: jeder darf es kopieren, verändern, weitergeben und für eigene Zwecke nutzen — auch kommerziell. Die einzige Bedingung ist, dass der Autor (Jürgen "George" Sawinski) in der Quellenangabe genannt wird.

Haftungsausschluss: Dieser Kurs wurde mit größter Sorgfalt erstellt. Dennoch übernimmt der Autor keine Gewähr für die Richtigkeit, Vollständigkeit oder Aktualität der Inhalte. Die Nutzung der Inhalte erfolgt auf eigene Verantwortung. Der Autor haftet nicht für Schäden, die aus der Nutzung oder der Unfähigkeit zur Nutzung der Inhalte entstehen.

Projekt-Webseite: `https://github.com/jsawinski/twin-c-course`

Teil I

Die Programmiersprache C

1
Ein paar Grundlagen

1.1 Das Binärsystem

Computer rechnen im sogenannten Binärsystem, sie kennen nur 0 und 1. Um größere Zahlen darstellen zu können, werden mehrere dieser Nullen und Einsen, genannt "**Bit**" aneinandergereit. Am gebräuchlichsten dabei ist das "**Byte**" (siehe Abb. 1.1), das aus 8 Bit besteht.
Jedem Bit wird nun ein Wert zugeordnet. Das erfolgt mathematisch gemäß:

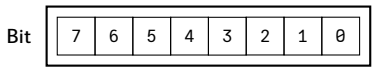


Abbildung 1.1: Ein Byte mit den Bit-Positionen 0 bis 7.

$$2^b$$

wobei b die Nummer des Bits ist. Das ist die sogenannte "Potenz", eine mathematische Operation[1] die sagt, wie oft eine Zahl mit sich selbst multipliziert wird. In diesem Fall wird die "2" b-mal mit sich selbst multipliziert, zum Beispiel:

$$\begin{aligned} 2^1 &= 2 && = 2 \\ 2^2 &= 2 \cdot 2 && = 4 \\ 2^3 &= 2 \cdot 2 \cdot 2 && = 8 \end{aligned}$$

Interessant ist nun die Frage, was 2^0 ist. Wenn Ihr genau hinschaut, dann merkt Ihr, dass, um in die nächste Zeile zu kommen, Ihr das Ergebnis mit 2 multipliziern müsst. Umgekehrt genauso: um in die vorhergehende Zeile zu kommen, muss man durch 2 teilen. Wenn also 2^1 zwei ist und man, um zu 2^0 zu kommen durch zwei Teilen muss, dann ergibt sich

$$2^0 = 1\,.$$

Übrigens, jede Zahl hoch 0 hat als Ergebnis die 1 (wir kommen später darauf zurück).[2]

Um auf das **Byte** zurückzukommen, heißt dass, dass den **Bits** jeweils die Werte 1 bis 128 zugeordnet sind (siehe Abb. 1.1).

[1] In der Schule habt Ihr wahrscheinlich den Begriff der Grundrechenarten (oder Grundrechnungsarten) kennengelernt. Der Begriff "Operation" bedeutet *Ausführungshandlung*, also ein Vorgang, bei dem aus einem oder mehreren Werten ein neuer Wert berechnet wird - oder in anderen Worten eine *Rechnungsart*. Die **Zeichen**, die diese Operationen beschreiben (wie **+**, **-**, **&**, **|**), nennt man **Operatoren**. Die Werte, mit denen eine Operation arbeitet, heißen **Operanden**. Zum Beispiel sind die Grundrechenarten, die Ihr in der Schule gelernt habt, also Addition, Subtraktion, Multiplikation und Division (Plus, Minus, Mal, Geteilt), sogenannte **arithmetische Operationen**.

[2] Es gibt ein Ausnahme und das ist $\mathbf{0^0}$. Wenn Ihr diese Rechnung auf dem Computer ausführt, dann erhaltet Ihr als Ergebnis höchstwahrscheinlich auch die **1**. Die Mathematik dahinter kommt allerdings nicht zu einem eindeutigen Ergebnis.

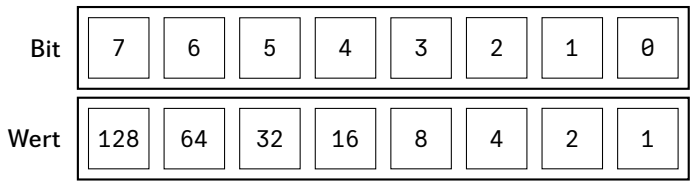


Abbildung 1.2: Ein Byte mit Bit-Positionen und den zugehörigen Werten (2^b).

Damit können wir nun die Zahlen 0 bis 255 darstellen, z.B.:

$$\begin{aligned}
&00000000 \rightarrow 0\,,\\
&00000011 \rightarrow 3\,,\\
&01011000 \rightarrow 88\,,\\
&11111111 \rightarrow 255\,.
\end{aligned}$$

Das Addieren funktioniert nun genauso, wie Ihr es aus der Schule, in der natürlich das Zehnersystem verwendet wird, gelernt habt. Beispiel:

```
  00100111       39
+ 00000110     +  6
     11          1     (Übertrag)
 ---------      ---
  00101101       45
```

Entsprechend lässt sich auch die Multiplikation durchführen:

```
00001010·00101010 =     10·42
        ..|.|.|.
  00000000.|.|.|.
   00000000|.|.|.
    00001010.|.|.
     00000000|.|.
      00001010.|.
       00000000|.
        00001010.
         00000000
         1 1             (Übertrag)
 ---------------
       110100100         420
```

Wenn Ihr genau hinseht, dann bemerkt Ihr, dass wir zwei Bytes multipliziert haben, aber das Ergebnis benötigt 9 Bits, also ein Bit mehr als ein Byte. Das führt zu dem Problem, dass, wenn Ihr auf einem Computer diese Multiplikation durchführt, das höchste Bit (die letzte "1" links) abgeschnitten wird und das Ergebnis, dass der Computer euch liefert, 164 ist. Das gleiche passiert bei einem Byte, wenn Ihr zu 255 die 1 dazuzählt, das Ergebnis ist 0 (in der Fachsprache als **Überlauf** bezeichnet).

Computer können auch negative Zahlen darstellen. Dazu nützt man aus, dass, wenn 255 + 1 zum Ergebnis 0 hat, dann hat 0 – 1 als Ergebnis –1, binär also 11111111. Dies nennt man das "Zweierkomplement" (den Begriff müsst Ihr euch natürlich nicht merken). Wichtig ist: das höchste

Bit zeigt, dass es sich um eine negative Zahl handelt. Damit können dann mit einem Byte die Zahlen -128 bis 127 dargestellt werden:

```
10000000        -128
10000001        -127
...
11111111        -  1
00000000           0
00000001           1
...
01111111         127
```

Wie wird nun aus einer positiven Zahl eine Negative? Das passiert, indem man alle Bits invertiert (das heißt, aus 0 wird 1 und aus 1 eine 0) und 1 addiert:

```
(-127)  10000001  ->  01111110
                     +00000001
                      --------
                      01111111     (127)
```

Das funktioniert auch umgekehrt, also aus einer negativen Zahl eine positive zu machen:

```
(127)   01111111  ->  10000000
                     +00000001
                      --------
                      10000001     (-127)
```

Das Interessante an diesem "Zweierkomplement" ist nun, dass man, z.B. eine Addition mit genau der gleichen Methode wie bei positiven Zahlen durchführen kann. Beispiel:

```
   00000101       5
 + 11111111      -1
   1111111       (Übertrag)
 -----------
   00000100       4
```

Und in der Tat werden im Prozessor des Computers Addition und Subtraktion von positiven und negativen Zahlen auf genau diese Weise durchgeführt, ohne dass der Prozessor speziell zwischen positiven und negativen Zahlen unterscheiden muss. Im nächsten Abschnitt lernt Ihr nun die Grundelemente, aus denen ein Computer, sowie genau diese Rechenoperationen, aufgebaut sind.

1.2 Logikgatter — die Bausteine des Prozessors

Im vorhergehenden Abschnitt haben wir gesehen, dass Computer im Binärsystem rechnen. Das bedeutet, dass sie nur mit den Zahlen 0 und 1 arbeiten. Aber wie kann ein Computer damit rechnen? Wie kann er zum Beispiel die Zahl 2 mit der Zahl 3 multiplizieren? Bevor wir das

verstehen, müssen wir uns anschauen, wie Computer aufgebaut sind: die zentrale Recheneinheit (oder der Prozessor, kurz CPU)[3] besteht aus vielen kleinen Schaltungen, die mit nur zwei Zuständen arbeiten: **0** und **1**. Diese Schaltungen nennt man **Logikgatter**. Die wichtigsten sind:

[3] CPU heißt **Central Processing Unit** (Zentrale Recheneinheit). Inzwischen haben Computer oft mehrere Recheneinheiten die für unterschiedliche Aufgaben zuständig sind. Wenn ihr zum Beispiel Computerspiele spielt, werden Grafikberechnungen von einer "GPU" (Graphics Processing Unit) durchgeführt. Darüber hinaus findet Ihr auch die NPU (Neural Processing Unit), die speziell für neuronale Netze, also künstliche Intelligenz (KI), optimiert ist.

UND-Gatter [4]

Das UND-Gatter liefert nur dann eine 1, wenn beide Eingänge 1 sind.

[4] Englisch: **and**

A	B	A and B
0	0	0
0	1	0
1	0	0
1	1	1

ODER-Gatter [5]

Das ODER-Gatter liefert eine 1, wenn mindestens einer der Eingänge 1 ist.

[5] Englisch: **or**

A	B	A or B
0	0	0
0	1	1
1	0	1
1	1	1

XOR-Gatter (exklusives ODER) [6]

Das XOR-Gatter liefert eine 1, wenn genau einer der Eingänge 1 ist.

[6] Englisch (kurz): **xor**

A	B	A xor B
0	0	0
0	1	1
1	0	1
1	1	0

NICHT-Gatter [7]

Das NICHT-Gatter invertiert den Eingang: aus 0 wird 1, aus 1 wird 0.

[7] Englisch: **not**

A	not A
0	1
1	0

Aus der Kombination dieser Logikgatter kann man dann auch komplexere Schaltungen bauen, die zum Beispiel alle Grundrechenarten (Addition, Subtraktion, Multiplikation und Division) von zwei Zahlen durchführen können.

Übung 1a: Wir haben oben die Addition von zwei binären Zahlen durchgeführt. Für die erste Stelle (ganz rechts) haben wir die beiden Bits addiert und das Ergebnis, aber auch den Übertrag notiert. Finde heraus, wie man die Addition der beiden Bits und den Übertrag mit den binären Operationen `AND`, `OR` und `XOR` berechnen kann.

Hinweise: Zur Lösung der Aufgabe ist es sinnvoll, eine Wahrheitstabelle für die Addition von zwei Bits und den Übertrag zu erstellen:

A	B	S	C
0	0	0	0
...	...	...	...

Hier steht das "S" steht für die Summe (Englisch "sum") und das "C" für den Übertrag (Englisch "carry").

(Lösung S. 63)

1.3 Bitweise Operationen in C

Die Logikgatter, die wir im vorhergehenden Abschnitt kennengelernt haben, sind die Bausteine, aus denen Computer aufgebaut sind. In C und C++, sowie vielen anderer Programmiersprachen, stehen diese als sogenannte **bitweise Operationen** zur Verfügung. Mit ihnen lassen sich die gleichen Operationen durchführen, wie mit den Logikgattern. Das bedeutet, dass Ihr mit diesen Operatoren auf den einzelnen Bits einer Zahl arbeiten könnt. Die Liste der bitweisen Operatoren in C und C++ ist:

& (and)
Beispiel: `1101 & 1011 = 1001`

| (or)
Beispiel: `1101 | 1011 = 1111`

^ (xor)
Beispiel: `1101 ^ 1011 = 0110`

~ (not)
Beispiel: `~1101 = 0010`

Darüber hinaus gibt es noch die sogenannten **Schiebe-Operatoren**[8], mit denen Ihr die Bits einer Zahl nach links[9] oder rechts[10] verschieben könnt. Das ist zum Beispiel nützlich, wenn Ihr eine Zahl mit 2 multiplizieren oder durch 2 teilen wollt. Die Shift-Operatoren sind:

<< (shift left)
Beispiel: `00000011 << 2 = 00001100`

[8] Englisch: **shift**
[9] Englisch: **left**
[10] Englisch: **right**

>> *(shift right)*
Beispiel: `11000101 >> 2 = 00110001`

Vermutlich werdet Ihr Euch jetzt fragen, wozu diese bitweisen Operationen überhaupt gut sind. Es gibt eine Vielzahl von Anwendungsmöglichkeiten, zum Beispiel werden wir später sehen, dass sich damit sehr einfach überprüfen lässt, ob eine Zahl gerade oder ungerade ist. Darüber hinaus kann auch eine Vielzahl von Ein- und Ausgabekanälen eines Computers sehr einfach angesteuert werden, ohne dass dafür komplexe Berechnungen notwendig wären. Es gibt aber viele Algorithmen[11], die sich mit nur mit bitweisen Operationen gut umsetzen lassen.

Übung 1b: Wir haben oben die Addition von zwei binären Zahlen an folgendem Beispiel durchgeführt:

```
  00100111       39
+ 00000110     +  6
     11          1     (Übertrag)
 ---------      ---
  00101101       45
```

Aus der letzten Übung wisst Ihr bereits, dass Ihr für das 0-te Bit (ganz rechts), aus den Zahlen *A* und *B* durch **xor** die Summe und durch **and** den Übertrag berechnen könnt. In Form eines Algorithmus lässt sich die das folgendermaßen darstellen:

```
S = A xor B
C = A and B
```

Versucht nun herauszufinden, welchen weiteren Schritte notwendig sind, um die Summe von $A + B$ zu berechnen.

(Lösung S. 63)

[11] Ein Algorithmus ist eine *genaue, endliche Abfolge von eindeutigen Anweisungen* zur Lösung eines Problems. Ein Kochrezept, eine Bauanleitung oder die Anleitung für ein Videospiel sind alles Algorithmen – in der Informatik sind sie die *Baupläne* für Programme.

2
Erste Schritte in C

2.1 Programm: Hallo Welt!

In alter Tradition ist das erste Programm, das man beim Erlernen einer neuen Programmiersprache zu sehen bekommt, gewöhnlich das "Hallo Welt!" Programm.[1] Da wir uns zunächst auf die Programmiersprache "C" beschränken, ist das:

[1] siehe Wikipedia 2003b

```c
/* Das "Hallo Welt"-Programm. */
#include <stdio.h>

int main() {
    printf("Hallo Welt!\n");
    return 0;
}
```

Erklärung:

`/* Das "Hallo Welt"-Programm. */`

Das ist ein **Kommentar**. Der "Compiler"[2] ignoriert ihn völlig. Er ist nur für uns Menschen da, um den Code zu erklären.

In C gibt es zwei Arten von Kommentaren:

- `/* ... */` — für längere Kommentare über mehrere Zeilen.
- `// ...` — für kurze Kommentare in einer Zeile.

[2] Der Compiler ist das Programm, das den C-Code in die sogenannte Maschinensprache, also ein Format, das der Prozessor direkt versteht, übersetzt.

`#include <stdio.h>`

Das ist eine **Header-Datei** (auf Deutsch auch *Kopfdatei*).

`stdio.h` steht für *Standard Input Output*. Sie enthält Informationen über Funktionen wie `printf`, die wir später benutzen wollen.

Merke: Wenn wir eine Funktion benutzen wollen, die wir nicht selbst geschrieben haben, müssen wir die dazugehörige Header-Datei einbinden.

```
int main() {
```

Das ist die **Hauptfunktion** (englisch: *main function*).

Jedes C-Programm startet hier. Der Computer führt den Code in den geschweiften Klammern `{ ... }` aus.

`int` bedeutet, dass die Funktion eine ganze Zahl (Integer) zurückgibt — das ist der Rückgabewert, der weiter unten kommt.

```
printf("Hallo Welt!\n");
```

Das ist die **Ausgabe**.

`printf` steht für *print formatted* und schreibt Text auf den Bildschirm (genauer: in die Konsole).

Das `\n` ist ein **Zeilenumbruch**. Es sorgt dafür, dass der Cursor nach der Ausgabe in die nächste Zeile springt.

Wichtig: Jede Anweisung in C endet mit einem Semikolon `;`.

```
return 0;
```

Das ist die **Rückgabe**.

Sie sagt dem Betriebssystem: *"Alles ist gut gelaufen."*

`0` bedeutet in der Regel *"kein Fehler"*. Wenn ein Programm mit `return 0;` endet, weiß der Computer, dass das Programm erfolgreich war.

```
}
```

Die schließende, geschweifte Klammer beendet die `main`-Funktion. Alles zwischen { und } gehört zur Funktion.

Merke:

- Ein C-Programm beginnt immer in der `main()`-Funktion.
- `printf()` schreibt Text auf den Bildschirm.
- `\n` macht einen Zeilenumbruch.
- `return 0;` sagt dem Computer: "Alles OK."
- Jede Anweisung endet mit `;`.

2.2 Programm: Zahlen

Um einen Computer dazu zu bringen, etwas zu tun, müssen wir ihm sagen, was er tun soll. Dazu brauchen wir Variablen. Eine **Variable** ist ein Platzhalter für einen Wert. Wir können ihr einen Namen geben und ihr einen Wert zuweisen. Beispiel: Wir wollen eine Variable mit dem Namen `zahl` erstellen und ihr den Wert 42 zuweisen. In C sieht das so aus:

```c
/* Deklaration einer Zahl. */
#include <stdio.h>

int main() {
  int zahl = 42;
  printf("Die Zahl: %d\n", zahl);
  return 0;
}
```

Erklärung:

`int zahl = 42;`

Das ist die **Deklaration**[3] (Festlegung des Namens und um was es sich handelt) und **Initialisierung**[4] (die erste Zuweisung eines Wertes) einer Variablen.

`int` ist der Datentyp, in diesem Fall eine ganze Zahl (Integer).
`zahl` ist der Name der Variable.[5]
`42` ist der Wert, der der Variable zugewiesen wird.

Es ist auch möglich, eine Variable ohne Initialisierung zu deklarieren:

```c
int zahl; // Deklaration ohne Initialisierung
```

Den Wert kann man ihr dann zu einem späteren Zeitpunkt zuweisen:

```c
zahl = 42; // Zuweisung eines Wertes
```

`printf("Die Zahl: %d\n", zahl);`

Das ist die **Ausgabe**.

`printf` steht für *print formatted* und schreibt Text auf den Bildschirm (genauer: in die Konsole).

Das `%d` ist ein Platzhalter[6] für eine ganze Zahl, die an dieser Stelle ausgegeben wird.

Das `\n` ist ein **Zeilenumbruch**. Es sorgt dafür, dass der Cursor nach der Ausgabe in die nächste Zeile springt.

[3] Eine **Deklaration** ist eine Mitteilung an den Compiler: "Es gibt etwas mit diesem Namen und diesem Typ". Auch ist `int main();` eine Deklaration, allerdings fehlt die Definition oder Implementation, also der Code.

[4] Die **Initialisierung** einer Variable ist die erste Zuweisung eines Wertes. Es ist generell empfehlenswert, jede Variable zu initialisieren (wenn man sich nicht sicher ist, nimmt man die `0`), da ansonsten die Variable einen zufälligen Wert haben kann (siehe dazu Kapitel 4.1).

[5] Variablenamen sind nicht beliebig. Es gibt eine ganze Reihe von Regeln, die man befolgen muss. Für den Anfang sollte folgendes gelten: Eine Variable beginnt mit einem Buchstaben (groß oder klein), danach folgen Buchstaben (groß und klein), Zahlen und das "underscore" (der Unterstrich _). Wenn Ihr es genauer wissen wollt, dann lest im Anhang nach (Kap. B.2.8).

[6] Um genau zu sein, es ist eine **Formatierungsanweisung**. Eine ausführliche Erklärung findet Ihr im Anhang unter `printf` (libc) auf S. 58.

Merke:

- Eine Variable wird deklariert und initialisiert. Dabei wird ihr ein Datentyp zugewiesen.
- `printf()` kann auch den Inhalt von Variable in von Menschen lesbarer Form ausgeben.

2.3 Programm: Fibonacci

Die Fibonacci-Zahlen[7] sind eine Folge von Zahlen, bei der jede Zahl die Summe der beiden vorhergehenden ist. Die Folge beginnt mit den Zahlen 0 und 1. Die ersten paar Fibonacci-Zahlen sind:

[7] siehe Wikipedia 2003a

$$
\begin{aligned}
F(0) &= 0,\\
F(1) &= 1,\\
F(2) &= 1,\\
F(3) &= 2,\\
F(4) &= 3,\\
F(5) &= 5,\\
F(6) &= 8,
\end{aligned}
$$

und so weiter. Die Fibonacci-Zahlen haben viele interessante Eigenschaften und Anwendungen in der Mathematik, der Natur und der Informatik. Sie tauchen in der Natur auf, zum Beispiel in der Anordnung von Blättern an einem Stängel, in der Verzweigung von Bäumen, in der Anordnung von Blütenblättern und in der Struktur von Tannenzapfen.

Die Fibonacci-Zahlen werden folgendermaßen berechnet: Die ersten beiden Zahlen der Folge sind 0 und 1. Jede weitere Zahl ist die Summe der beiden vorhergehenden Zahlen. Hier ist ein Beispielprogramm, das die ersten 16 Fibonacci-Zahlen berechnet und ausgibt (die 0 als Startwert lassen wir weg).

```c
/* Die Fibonacci-Sequenz. */
#include <stdio.h>

int main() {
    int a = 0;
    int b = 1;

    for (int i = 1; i <= 16; i = i + 1) {
        int c = a + b;
        a = b;
        b = c;

        printf("Fibonacci %d: %d\n", i, a);
    }
    return 0;
}
```

Erklärung:

`int a = 0;`

Wir initialisieren die erste Fibonacci-Zahl mit 0.

`int b = 1;`
Wir initialisieren die zweite Fibonacci-Zahl mit 1.

`for(int i = 1; i <= 16; i = i+1) { … }`
Wir starten eine Schleife, die 16 Mal durchläuft, um die ersten 16 Fibonacci-Zahlen zu berechnen. Wisst Ihr übrigens, dass es in diesem Buch auch ein ausführliches Nachschlagewerk gibt? Dort könnt Ihr nachlesen, wie Schleifen genau funktionieren — schaut mal in Abschnitt B.2.6 nach.

`int c = a + b;`
Wir berechnen die nächste Fibonacci-Zahl, indem wir die beiden vorhergehenden Zahlen addieren.

`a = b;`
Wir verschieben die Werte: Die aktuelle zweite Zahl wird zur ersten Zahl

`b = c;`
und die neu berechnete nächste Zahl wird zur zweiten Zahl.

`printf("Fibonacci %d: %d\n", i, a);`
Wir geben die aktuelle Fibonacci-Zahl auf dem Bildschirm aus. Das `%d` ist ein Platzhalter für die aktuelle Zahl, die an dieser Stelle ausgegeben wird.

Merke:

- Die `for`-Schleife ermöglicht es einem, Code gemäß einer Bedingung beliebig oft zu wiederholen.

2.4 Programm: Reihen

Die Zahl π beschreibt das Verhältnis von Umfang zu Durchmesser eines Kreises. Sie ist eine sehr wichtige Zahl in der Mathematik, der Physik und Technik. Sie lässt sich aus einer unendlichen Reihe berechnen:

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \ldots$$

In der Praxis kann man natürlich aber nur endlich viele Terme[8] berechnen. Je mehr Terme man berechnet, desto genauer wird das Ergebnis. Im folgenden Programm wird diese Summe mit 1000000 Schritten berechnet (auf der Konsole wird π ausgegeben, indem man obige Summe einfach vervierfacht):

[8] Ein Term ist ein einzelner Baustein einer Summe — hier also jeweils einer der Brüche wie 1, $\frac{1}{3}$, $\frac{1}{5}$ usw. Unendlich viele Terme kann man natürlich nicht berechnen, weil das Programm sonst nie fertig würde.

```c
/* Berechnung von Pi/4 aus der Reihe 1 - 1/3 + 1/5 - 1/7 + ... */
#include <stdio.h>

int main() {
    double pi = 1.0;
    for (int i = 1; i <= 1000000; i++) {
        double n = 2 * i + 1;  // 3, 5, 7, 9, ...
        n = 1.0 / n;           // 1/n

        // Ausgabe der Zwischenergebnisse
        printf("%9d Pi: %.15f\n", i, 4 * pi);

        if (i & 1)    // Bedingung: i ungerade?
            pi -= n;  // ja (true)
        else
            pi += n;  // nein (false)
    }
    return 0;
}
```

Übung 2a: Einige von Euch haben sich vielleicht schon ein paar Stellen von π gemerkt und werden feststellen, dass das Ergebnis sechs Stellen nach dem Komma abweicht. Ihr könnt einmal das `printf()` durch folgernde Anweisung tauschen, die Euch die Abweichung von π anzeigt:

```c
printf("%9d Pi: %.15f  (% .15f)\n", i, 4*pi, 4*pi -
    3.14159265358979323846);
```

Wie groß wäre die Abweichung nach 1000 Schritten?

(Lösung S. 65)

Erklärung:

`double pi = 1.0;`

Bisher haben wir, um mit einer Zahl zu arbeiten, `int` benutzt. Um π zu berechnen, muss man jedoch mit Kommazahlen rechnen, um genau zu sein mit einer sogenannten "Gleitkommazahl".[9] Dies erreicht man in C mit `double` (man sagt, `pi` hat den Datentyp `double`). Es gibt eine ganze Menge von Datentypen, die Ihr noch kennenlernen werdet (Neugierige können das im Abschnitt B.2.2 nachlesen).

[9] Der Begriff *Gleitkommazahl* kommt daher, dass die Recheneinheit des Computers nicht unbegrenzt viele Stellen nach dem Komma darstellen kann, so wie ihr es auf einem Blatt Papier machen könntet. Stattdessen "merkt" sich der Computer nur eine Teil der Stellen und speichert dazu die Stelle, an der das Komma steht.

`for (int i = 1; i <= 1000000; i++)`

Das ist eine Schleifenanweisung, die wir schon im vorhergehenden Kapitel kennengelernt haben. Neu ist hier allerdings, wie *i* gezählt wird. Der Operator `++` sagt, dass *i* um eins erhöht werden soll (also die Kurzform von `i = i + 1`). Wie er genau funktioniert, werden wir aber später noch kennen lernen.

`double n = 2*i + 1`

Oben wurde beschrieben, wie man π aus einer Summe berechnen kann. Wenn Ihr nochmal genau hinschaut, dann tauchen in der Reihe immer nur die ungeraden Zahlen 3, 5, 7, usw. auf. Unser Zähler *i* liefert jedoch die Zahlen 1, 2, 3, ... Die Rechnung $2 \cdot i + 1$ liefert nun genau diese ungeraden Zahlen. Da wir π aber mit einer Gleitkommazahl berechnen muss auch das `n` eine Gleitkommazahl sein.

`n = 1.0 / n`

Was wir addieren oder subtrahieren müssen, ist aber nicht 3, 5, 7, usw. sondern 1/3, 1/5, 1/7, usw., wir bilden also hier den Kehrwert $n \leftarrow \frac{1}{n}$ (in Worten: *n* wird der Kehrwert $\frac{1}{n}$ zugeordnet).

if (i & 1)

Da wir abwechselnd addieren und subtrahieren müssen, muss eine Entscheidung getroffen werden, was von beidem stattfindet. Das macht man mit dem `if`:

```
if (Bedingung) {
    // Befehle, die ausgeführt werden,
    // wenn die Bedingung "wahr" (true) ist.
}
else {
    // Befehle, die ausgeführt werden,
    // wenn die Bedingung "falsch" (false) ist.
}
```

Bemerkt, dass wir die geschweiften Klammern im Code weglassen konnten, weil es nur ein einziger Befehl ist (das gleiche gilt übrigens auch für die `for`-Schleife).

Die Bedingung ist nun `i & 1`. Wenn ihr Euch an das Kapitel über "Bitweise Operationen" (S. 7) erinnert, dann prüft diese Bedingung ob das 0-te Bit eine 1 ist. Das ist genau dann der Fall, wenn *i* ungerade ist. Bildlich sieht das so aus (wir schauen uns dabei nur die untersten 4 Bits an):

```
      i       i & 1
0     0000    0000
1     0001    0001
2     0010    0000
3     0011    0001
4     0100    0000
...
```

Merke:

- Für Gleitkommazahlen verwenden wir den Datentyp `double`.
- Mit bitweisen Operationen kann man z.B. feststellen, ob eine Zahl gerade oder ungerade ist.

2.5 Programm: Lauflicht

Im folgenden C-Programm wird ein sogenanntes "Lauflicht" erzeugt. Dabei wird ein Sternchen von links nach rechts über den Bildschirm bewegt. Dabei werden wir eine neue Eigenschaft der sogenannten *Konsole* kennenlernen. Bisher habt ihr gesehen, dass man mit `printf()` Text auf den Bildschirm ausgeben kann. Um eine neue Zeile anzufangen, habe ihr den sogenannten Zeilenumbruch `\n` verwendet. In diesem Programm werden wir jedoch sehen, wie man den Cursor an den Anfang der aktuellen Zeile zurücksetzt, ohne eine neue Zeile zu beginnen: das geschieht mit `\r`.[10] Darüber hinaus werden wir eine Schleife verwenden, so dass das Programm unendlich lange läuft, bis es mit `Strg+C` abgebrochen wird:

```c
#include <stdio.h>

int main() {
    while(1) {
        for (int i = 0; i < 20; i++) {
            printf("\r");
            for (int j = 0; j < 20; j++) {
                if (i == j)
                    printf("*");
                else
                    printf(" ");
            }
        }
    }

    return 0;
}
```

Hier ist `while` eine neue Schleife:

```
while (Bedingung) {
    // Schleifenkörper
}
```

Dabei wird der *Schleifenkörper* so lange ausgeführt, wie die *Bedingung* wahr ist.

Wenn Ihr dieses Programm nun ausprobiert, stellt Ihr fest, dass fast nichts passiert. Woran liegt das? Das liegt daran, dass die Zeichen zwischengespeichert werden, man nennt das **puffern** (Englisch: *buffer*), bis ein Zeilenvorschub (`\n`) erscheint. Um sicherzustellen, dass alle Zeichen immer ausgegeben werden, muss man dem Computer sagen, dass er, auch ohne `\n` alles ausgeben soll. Das geschieht mit dem Kommando `fflush(stdout)`.

Aber selbst dann, wenn Ihr dieses Kommando verwendet, ist der

[10] Das `\r` steht für *Carriage Return* (deutsch: Wagenrücklauf). Der Begriff stammt von der mechanischen Schreibmaschine, die man früher zum Schreiben von Briefen benutzt hat (vielleicht habt Ihr ja mal eine im Museum oder bei den Großeltern gesehen): Dort beförderte der Wagenrücklauf den Schreibwagen zurück an den linken Rand, ohne die Zeile zu wechseln – genau das macht `\r` auch auf der Konsole: Der Cursor springt an den *Anfang der aktuellen Zeile*, der bereits geschriebene Text bleibt stehen und kann überschrieben werden. Neben `\r` und `\n` gibt es noch ein paar weitere *Escape-Sequenzen*, mit denen sich die Ausgabe steuern lässt. In der Tat stammen diese Sequenzen aus einer Zeit, in der Computer noch keine Bildschirme hatten und die Ausgabe hauptsächlich über Drucker erfolgte – angelehnt an die mechanische Schreibmaschine. Ein weiteres interessantes Beispiel ist übrigens `\a`, das für einen Alarmton steht

Ablauf viel zu schnell. Das Programm muss also "gebremst" werden und das macht man indem man eine Pause einbaut. Dazu verwendet Ihr

```
#define __GNU_SOURCE
#include <unistd.h>

usleep(100000);
```

Das hier eine neue Kopfdatei ("*Header*") verwendet wird, seht Ihr sofort. Um den Befehl `usleep()` verwenden zu können, ist das `#define __GNU_SOURCE` vor dem Einbinden der Kopfdatei notwendig – wir kommen im nächsten Abschnitt auf die genaue Bedeutung zurück. Damit sieht das vollständige Programm folgendermaßen aus:[11]

```
#define __GNU_SOURCE
#include <stdio.h>
#include <unistd.h>

int main() {
    // Das Lauflicht wird so lange wiederholt,
    // bis man "Strg-C" drückt.
    while (1) {
        // Zähler für das Lauflicht (Breite von 20 Zeichen)
        for (int i = 0; i < 20; i++) {
            // An den Zeilenanfang zurückkehren
            printf("\r");

            // Ausgabe eines Leerzeichens oder Sternes
            // (genau da, wo i == j ist, ein Stern)
            for (int j = 0; j < 20; j++) {
                if (i == j)
                    printf("*");
                else
                    printf(" ");
            }

            // Zeichen ausgeben (Puffer leeren)
            fflush(stdout);

            // Pause von 0.1 Sekunden
            // (ein Durchlauf braucht also etwa 2 Sekunden)
            usleep(100000);
        }
    }

    return 0;
}
```

[11] Anders als in den vorhergehenden Kapiteln ist das Programm vollständig kommentiert. Bei so einem kurzen Programm könnt Ihr natürlich die Kommentare weglassen. Allerdings ist es gut, jeden Programmcode zu kommentieren, so dass man später leicht wieder erkennen kann, was an welcher Stelle gemacht wird.

Merke:

- Die Ausgabe mit `printf` ist gepuffert. Mit dem Kommando `fflush(stdout)` kann man dem Computer sagen, dass er alle gepufferten Zeichen ausgeben soll.
- Mit dem Befehl `usleep` kann man in einem Programm eine Pause machen (in `usleep` steht das "u" übrigens für Mikrosekunde und "sleep" heißt auf Deutsch "schlafen").

2.6 Programm: Lauflicht - Vertikal

Im letzten Kapitel haben wir ein Lauflicht gebaut, das waagerecht über den Bildschirm läuft. Jetzt wollen wir dasselbe senkrecht machen – ein Tropfen soll von oben nach unten fallen. Dabei lernen wir zwei neue Dinge kennen: Makros und Funktionen. (Funktionen schauen wir uns hier nur kurz an – wie man sie selbst schreibt, sehen wir im nächsten Kapitel.)

In unserem Programm aus dem vorhergehenden Kapitel stand an mehreren Stellen die Zahl 20: einmal in der `for`-Schleife für `i`, einmal in der `for`-Schleife für `j`. Wenn wir jetzt die Höhe des Lauflichts ändern wollen, müssten wir beide Stellen anpassen – wenn so etwas häufiger vorkommt, kann man das leicht vergessen. Hier kommen Makros ins Spiel. Ein Makro ist ein Name für einen Textbaustein, den der Compiler vor dem eigentlichen Übersetzen einsetzt. In C definiert man ein Makro mit `#define`:

```
#define HOEHE 20
```

Ab jetzt ersetzt der Compiler jedes HOEHE im Code durch 20, bevor er das Programm übersetzt.[12] Wichtig: Ein Makro ist kein Wert im Sinne einer Variable. Es gibt keinen Speicherplatz, keine Typprüfung, keine Rechenoperation. Es ist reine Textersetzung – der Compiler macht aus HOEHE buchstäblich 20, bevor er den Rest liest. (Deshalb schreibt man Makronamen üblicherweise in GROSSBUCHSTABEN, damit man sie im Code sofort von Variablen unterscheiden kann.)

[12] Das erste Makro, das Ihr schon gesehen habt, war `__GNU_SOURCE` in der ersten Zeile vor den Kopfdateien. Wenn Ihr nachschaut, werdet Ihr feststellen, dass es sich um ein *leeres* Makro handelt, das im Code einfach nur dazu benutzt wird, dem Compiler zu sagen: „Ich möchte auch Funktionen benutzen, die nicht zum Standard gehören" – zum Beispiel `usleep()`.

Um das Programm übersichtlicher zu machen, führen wir zwei Funktionen ein:

```
void konsole_start() {
    printf("\x1b[H");
}

void tropfen(int b) {
    if (b)
        printf("*");
    else
        printf(" ");
}
```

Funktionen fassen mehrere Anweisungen zu einem benannten Block zusammen, den man immer wieder aufrufen kann. Die beiden Funktionen haben folgende Eigenschaft:

konsole_start()

Diese Funktion gibt eine sogenannte *ANSI-Escape-Sequenz*[13] aus. Diese Sequenzen beginnen meistens mit `\x1b[`, gefolgt von einem Konsolenbe-

[13] Wikipedia 2026

fehl. In diesem Fall ist es das `H`, das für "home" steht: Der Cursor geht dabei zur Position "Zeile 1, Spalte 1". Wir werden in den folgenden Kapiteln noch weitere Möglichkeiten kennenlernen.

tropfen(int b)

Diese Funktion gibt einen "Tropfen" auf dem Bildschirm aus, wenn man für den Parameter[14] `b` eine von `0` verschiedene Zahl übergibt. Ansonsten wird ein Leerzeichen ausgegeben. Damit könnt Ihr nun im Hauptteil des Programms die innere `for`-Schleife (ab Zeile 16, S. 19) durch

```
    for (int j = 0; j < 20; j++)
        tropfen((i == j))
```

ersetzen. Euch sollte nun das `(i == j)` auffallen. In C (und vielen anderen Programmiersprachen) haben Vergleiche ein Ergebnis, das man einer Variable zuordnen kann. In C ist das Ergebnis ein `int` und `(i == j)` liefert, wenn `i` und `j` gleich sind, den Wert `1` (für *wahr*), ansonsten ist das Ergebnis `0` (für *falsch*).

[14] Ein Parameter ist ein Platzhalter für einen Wert, den man einer Funktion beim Aufruf mitgibt. In `tropfen(int b)` heißt der Parameter `b` – beim Aufruf `tropfen(1)` wird `b` zu `1`, beim Aufruf `tropfen(0)` wird `b` zu `0`. Der Name `b` ist dabei egal (er könnte auch `wert` oder `x` heißen); wichtig ist nur, an welcher Stelle der Wert steht. (Manchmal sagt man statt Parameter auch Argument – für den Anfang reicht es, wenn Ihr Euch einen von beiden Begriffen merkt.)

Wenn Ihr das Programm aus dem letzten Kapitel nehmt und die Änderungen von oben einbaut, sieht es folgendermaßen aus:

```
#define __GNU_SOURCE
#include <stdio.h>
#include <unistd.h>

#define HOEHE 20

void konsole_start() {
    printf("\x1b[H");
}

void tropfen(int b) {
    if (b)
        printf("*");
    else
        printf(" ");
}

int main() {
    while (1) {
        for (int i = 0; i < HOEHE; i++) {
            printf("\r");

            for (int j = 0; j < HOEHE; j++) {
                tropfen((i == j));
            }

            fflush(stdout);
            usleep(100000);
        }
    }

    return 0;
}
```

Nun könnte Ihr überlegen, wie Ihr daraus das “vertikale Lauflicht” macht, bevor Ihr Euch die folgende Musterlösung anschaut.

Musterlösung

```c
#define __GNU_SOURCE
#include <stdio.h>
#include <unistd.h>

// Makro für die Anzahl der Zeilen, die das Lauflicht laufen soll
#define HOEHE 20

// Hilfsfunktion: springe in der Konsole in die 1. Zeile und 1. Spalte
void konsole_start() {
    printf("\x1b[H");
}

// Hilfsfunktion: Ausgabe eines "Tropfens" oder eines Leerzeichens
void tropfen(int b) {
    if (b)
        printf("*");
    else
        printf(" ");
}

int main() {
    // Das Lauflicht laeuft, bis es durch Strg+C abgebrochen wird
    while (1) {
        // Schleife i: Zeile, in der der "Tropfen" erscheinen soll
        for (int i = 0; i < HOEHE; i++) {
            konsole_start();

            // Ausgabe der Leerzeichen oder "Tropfen"
            for (int y = 0; y < HOEHE; y++) {
                tropfen((y == i));
                printf("\n");
            }

            // Puffer leeren
            fflush(stdout);

            // Kleine Pause
            usleep(100000);
        }
    }

    return 0;
}
```

2.7 Programm: Ein Würfel

Ein Würfel liefert ja bekanntlich zufällige Zahlen von 1 bis 6. Um so etwas zu programmieren, brauchen wir Zufallszahlen – und die kann der Computer auf Befehl liefern:

```
#include <stdlib.h>

int rand();
```

Der Befehl `rand()`, definiert in der Kopfdatei "stdlib.h" (*standard library*, so viel wie "Standardbibliothek"), gibt Euch Zufallszahlen vom Typ `int`.

Übung 2b: Schreibt ein Programm, das eine Zufallszahl auf dem Bildschirm ausgibt. Ihr könnt dazu einfach Euer Programm aus Abschnitt 2.2 kopieren und die 42 durch `rand()` ersetzen. Vergesst aber nicht, die Kopfdatei `stdlib.h` einzubinden. Was fällt Euch auf, wenn Ihr das Programm mehrmals aufruft? Probiert auch aus, 5 Zufallszahlen mit einer Schleife auszugeben.

(Lösung S. 65)

In der Aufgabe habt Ihr gesehen, dass die generierten Zufallszahlen immer mit derselben Zahl anfangen und darüber hinaus auch die gleiche Reihenfolge aufweisen. Das liegt daran, dass *echte* Zufallszahlen sehr schwer herzustellen sind.[15] Daher wurden mathematische Algorithmen entwickelt um eine Zahlenfolge zu berechnen, die sich so verhält, als wäre sie zufällig. Dabei ist es möglich, den Anfangswert festzulegen (der sogenannte *seed*, englisch für "Saatkorn" – gemeint ist der Startwert der Zahlenfolge). Um nun beim Programmstart einen neuen Anfangswert zu setzen, behilft man sich im Allgemeinen damit, die aktuelle Zeit zu nehmen. Dazu schreibt man sich die Funktion:

```
#include <stdlib.h>
#include <time.h>

void random_init() {
    srand(time(NULL));
}
```

Hier ist `srand(seed)` die Funktion, mit der man den Anfangswert der Zufallszahlenfolge setzt und `time(NULL)` die Anzahl der Sekunden seit dem 1.1.1970 (dafür benötigen wir die Kopfdatei `time.h`, siehe auch Funktionsreferenze S. 60).

Wenn wir einen Würfel programmieren wollen, der z.B. die Zahlen 1 bis 6 liefert, dann müssen wir noch einen geeigneten Weg finden, das Ergebnis von `rand()` auf diesen Bereich einzuschränken. Wir verwenden hier die Operation **modulo**, in C/C++ wird dafür der Operator `%` benutzt.[16] Die Funktion `rand()` liefert nun die Zahlen 0 bis `RAND_MAX` (siehe

[15] Selbst viele moderne Computer haben keine echten Zufallszahlengeneratoren. Die Betriebssysteme helfen sich jedoch damit, Zufallszahlen aus dem Benutzerverhalten (z.B. durch Tastatureingabe) zu generieren.

[16] Das "modulo" ist der Rest einer Division, Beispiel: 10 ÷ 3 = 3 Rest 1.

Lösung der obigen Aufgabe; Funktionsreferenz S. 59). Ihr könnt nun einmal ausprobieren, was passiert, wenn Ihr in Eurem Programm die Zeile

```
    int zahl = rand();
```

durch

```
    int zahl = rand() % 6;
```

ersetzt. Wenn Ihr das nun ausprobiert, merkt Ihr, dass die Zahlen von 0 bis 5 ausgegeben werden – das heißt, wir müssen noch 1 dazu addieren, um die Zahlen von 1 bis 6 zu erhalten.

Da wir solche Zufallszahlen aber noch in anderen Programmen nutzen wollen, ist es ratsam, eine Funktion zu schreiben, bei der man festlegen kann, was der kleinste und was der größte Wert ist.

Einschub: *Funktionen*

Im ersten Programm (“Hallo Welt!”, S. 2.1) habt ihr zwei Funktionen kennengelernt. Zum einen das `int main()` und zum anderen das `printf`. Der Unterschied ist, `main` habt Ihr selbst geschrieben, `printf` dagegen war hinter einer Kopfdatei (genauer: `stdio.h`) “versteckt”. Wir haben auch schon im letzten Kapitel weitere eigene Funktionen eingeführt: `konsole_start()` und `tropfen()`. Der Aufbau ist in allen Fällen der gleiche:

```
Rückgabetyp Funktionsname ( Parameter ) {
    Funktionskörper
}
```

Der *Rückgabetyp* kann jeder Datentyp sein, den C kennt – zum Beispiel `int` oder `double`. Wenn man eine Funktion programmieren will, die nichts zurück gibt (wie oben das `random_init()`), dann verwendet man `void` (deutsch: “Leere” oder “Nichts”).

Den *Funktionsnamen* könnt Ihr selbst wählen. Es gibt aber eine ganze Reihe von Regeln, die man beachten muss. Für den Anfang solltet Ihr Euch einfach an folgende Regel halten: verwendet einen Funktionsnamen der aus ganzen Wörtern besteht (kleingeschrieben) und durch Unterstriche verbunden ist, zum Beispiel: `diese_funktion_macht_etwas()`.[17]

Die *Parameter* sind die “Eingangswerte” der Funktion. In der Funktion verhalten sie sich wie normale Variablen. Sie werden als Liste übergeben – in C getrennt durch Kommas. Jeder Parameter besteht aus einem Datentyp und einem Variablennamen, so wie Ihr es schon von der Deklaration von Variablen kennt. Erinnert Ihr Euch an `tropfen(int b)` aus dem letzten Kapitel? Jetzt versteht Ihr, was da passiert ist: `b` war der Parameter, `void` der Rückgabetyp (weil die Funktion nichts zurückgibt), und der Code in den geschweiften Klammern war der Funktionskörper.

[17] In der Informatik, so wie auch in der Wissenschaft und vielen anderen Bereichen, hat sich die englische Sprache durchgesetzt. Wenn Ihr bereits Englisch lernt und wisst, wie die Funktion auf Englisch geschrieben werden könnte, dann verwendet Englisch. Im genannten Beispiel wäre das `this_function_does_something()`.

Wenn man einen Wert zurückgeben will, benutzt man `return`. Ein Beispiel:

```
int erhoehe(int n) {
    return n+1;
}
```

Diese Funktion gibt den Wert `n+1` zurück. Wichtig ist: mit dem `return` kehrt die Funktion zurück, sie ist zu Ende – jeglicher Code nach `return` wird nicht mehr ausgeführt.

Damit können wir nun wieder zu den Zufallszahlen zurückkehren. Für den Würfel und folgende Programme brauchen wir eine Funktion, die Zufallszahlen zwischen zwei Werten liefert. Die beiden Werte nennen wir `min` (für "Minimum", das heißt die kleinste Zahl) und `max` (für "Maximum", das heißt die größte Zahl). Zunächst fangen wir mit dem obigen Beispiel an:

```
int zahl = rand() % max;
```

Wie Ihr festgestellt habt, liefert das *modulo* als Ergebnis eine Zufallszahl zwischen `0` und `max-1`. Für einen normalen Würfel (einer der mit 1 beginnt und bei, zum Beispiel 20 endet) müssten wir also noch eine `1` addieren. Was passiert nun aber mit dem `min`?

Übung 2c:

a) Die Berechnung hängt eng mit dem "Zaunpfahlproblem" zusammen: wenn ihr einen Zaun bauen wollt, der alle Meter einen Zaunpfahl haben soll, wie viele Zaunpfähle braucht man dann für 10 m Zaun?

b) Ein Architekt sagt Euch, der Zaun beginnt 6 m neben dem Haus und endet in einer Entfernung von 10 m vom Haus. Wie lange ist der Zaun und wie viele Zaunpfähle braucht Ihr?

c) Wenn Ihr die vorhergehenden Teilaufgaben gelöst habt, dann könnt Ihr sicher auch beantworten, warum das folgende Beispiel falsch ist, wenn Ihr alle Zahlen von `min` bis `max` haben wollt:

```
int random_number(int min, int max) {
    return min + rand() % max;
}
```

Findet heraus, wie die Funktion richtig geschrieben werden muss.

(Lösung S. 66)

Damit haben wir nun alle notwendigen Hilfsfunktionen. Versucht nun

einmal, Euer Würfelprogramm selbst zu vervollständigen.

Musterlösung

Damit lässt sich nun das vollständige Programm schreiben:

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Hilfsfunktion, um den Zufallszahlengenerator zu initialisieren:
void random_init() {
    srand(time(NULL));
}

// Hilfsfunktion, um eine Zufallszahl im Bereich von min bis max zu
    erzeugen:
int random_number(int min, int max) {
    return min + rand() % (max - min + 1);
}

int main() {
    random_init();

    // Zufallszahl von 1 bis 6
    int zahl = random_number(1, 6);

    // Ausgabe auf dem Bildschirm
    printf("%d\n", zahl);

    return 0;
}
```

2.8 Programm: Das Spielfeld

Nachdem Ihr so viel Neues gelernt und kennengelernt habt, ist es nun Zeit, ein wenig zu Spielen. In diesem Kapitel soll das, was Ihr in Kapitel 2.5 und 2.6 gebaut habt, in beide Richtungen funktionieren – also sowohl waagerecht als auch senkrecht. Wir fangen damit an, ein Zeichen an einer zufälligen Stelle auf dem Bildschirm auszugeben – also nicht mehr nur auf einer festen Linie, sondern irgendwo im Feld. Damit legt Ihr die Grundlage für die Regensimulation, die wir später bauen.

Das Grundgerüst Eures Progamms mit den bekannten Hilfsfunktionen ist:

```
#define __GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

#define BREITE 60  // Breite des "Spielfeldes"
#define HOEHE 30   // Höhe des "Spielfeldes"

// ================================================================
// Zufallszahlen
// ================================================================

/* Initialisierung des Zufallszahlengenerators
 */
void random_init() {
    srand(time(NULL));
}

/* Berechnung einer Zufallszahl zwischen min und max.
 */
int random_number(int min, int max) {
    return min + rand() % (max-min+1);
}

// ================================================================
// Konsole
// ================================================================

void konsole_loeschen() {
    printf("\x1b[2J");
}

void konsole_start() {
    printf("\x1b[H");
}

// ================================================================
```

```c
// MAIN
// ================================================================

int main() {
    // Euer Code
    return 0;
}
```

Musterlösung

```c
#define __GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

#define BREITE 60  // Breite des "Spielfeldes"
#define HOEHE 30   // Höhe des "Spielfeldes"

//
    ================================================================================

// Zufallszahlen
//
    ================================================================================


/* Initialisierung des Zufallszahlengenerators
 */
void random_init() {
    srand(time(NULL));
}

/* Berechnung einer Zufallszahl zwischen min und max.
 */
int random_number(int min, int max) {
    return min + rand() % (max-min+1);
}

//
    ================================================================================

// Konsole
//
    ================================================================================


void konsole_loeschen() {
    printf("\x1b[2J");
}

void konsole_start() {
    printf("\x1b[H");
}

//
    ================================================================================

// Spielfeld
```

```c
//
    ============================================================================


struct spielfeld {
    char daten[HOEHE][BREITE + 1];  // +1 für '\0' am Ende jeder Zeile
    int breite;
    int hoehe;
};

void spielfeld_anzeigen(struct spielfeld* feld) {
    for (int y = 0; y < feld->hoehe; y++) {
        printf("%s\n", feld->daten[y]);
    }
}

void spielfeld_setzen(struct spielfeld* feld, int x, int y, char c) {
    if (x >= 0 && x < feld->breite && y >= 0 && y < feld->hoehe) {
        feld->daten[y][x] = c;
    }
}

void spielfeld_zufaellig_fuellen(struct spielfeld* feld, char c, int
    anzahl) {
    for (int i = 0; i < anzahl; i++) {
        int x = random_number(0, feld->breite - 1);
        int y = random_number(0, feld->hoehe - 1);

        /* Sicherstellen, dass das Feld frei ist */
        while (feld->daten[y][x] != ' ') {
            x = random_number(0, feld->breite - 1);
            y = random_number(0, feld->hoehe - 1);
        }

        feld->daten[y][x] = c;
    }
}

//
    ============================================================================

// MAIN
//
    ============================================================================


int main() {
    struct spielfeld feld;

    /* Spielfeld initialisieren */
    feld.breite = BREITE;
```

```
    feld.hoehe = HOEHE;
    for (int y = 0; y < HOEHE; y++) {
        for (int x = 0; x < BREITE; x++) {
            feld.daten[y][x] = ' ';
        }
        feld.daten[y][BREITE] = '\0';  // Nullterminator für jede Zeile
    }

    konsole_loeschen();

    // Alle 0.1s wird ein Spielfeld mit einem '*' gefüllt.
    while (1) {
        // Ein zufälliges freies Feld auswählen
        int x, y;

        x = random_number(0, BREITE - 1);
        y = random_number(0, HOEHE - 1);

        while (feld.daten[y][x] != ' ') {
            x = random_number(0, BREITE - 1);
            y = random_number(0, HOEHE - 1);
        }

        feld.daten[y][x] = '*';

        // Spielfeld anzeigen
        konsole_start();
        for (int y = 0; y < HOEHE; y++) {
            printf("%s\n", feld.daten[y]);
        }

        fflush(stdout);
        usleep(100000);
    }

    return 0;
}
```

2.9 Warnungen des Compilers

FIXME

```c
/* Warnungen des Compilers */
#include <stdio.h>

int main() {
  int zahl;
  printf("Die Zahl: %d\n", zahl);
  return 0;
}
```

```
10_warnungen.c: In Funktion »main«:
10_warnungen.c:6:3: Warnung: »zahl« ist nicht initialisiert,
                    wird aber benutzt [-Wuninitialized]
    6 |   printf("Die Zahl: %d\n", zahl);
      |   ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~
10_warnungen.c:5:7: Anmerkung: »zahl« wurde hier deklariert
    5 |   int zahl;
      |       ^~~~
```

Erklärung:

Merke:

3
Etwas Physik

3.1 Bewegung

Sicherlich habt Ihr schon einiges über Bewegung erfahren.

FIXME Wenn Ihr zum Beispiel im Zug sitzt Ihr fahrt mit $100\,\mathrm{km\,h^{-1}}$, dann bewegt sich das Auto mit dieser Geschwindigkeit relativ zur Straße.

4
Regensimulation in C

4.1 Warnungen des Compilers

FIXME

```c
/* Warnungen des Compilers */
#include <stdio.h>

int main() {
  int zahl;
  printf("Die Zahl: %d\n", zahl);
  return 0;
}
```

```
10_warnungen.c: In Funktion »main«:
10_warnungen.c:6:3: Warnung: »zahl« ist nicht initialisiert,
                    wird aber benutzt [-Wuninitialized]
    6 |   printf("Die Zahl: %d\n", zahl);
      |   ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
10_warnungen.c:5:7: Anmerkung: »zahl« wurde hier deklariert
    5 |   int zahl;
      |       ^~~~
```

Erklärung:

Merke:

Teil II

Die Programmiersprache C++

Teil III

Anhang

A
Software

Als Programmierumgebung wurde VSCode verwendet. Zusätzlich ist es empfehlenswert, die Erweiterung “C C++ Toolkit” zu installieren. Diese Erweiterung bietet unter anderem Syntaxhervorhebung, Autovervollständigung und Debugging-Funktionen für C und C++. Die Möglichkeiten, die VScode bietet, sind sehr umfangreich. Daher ist es empfehlenswert, wenn ihre jemanden fragt, der sich mit VScode auskennt, um die Umgebung optimal einzurichten.

B
Leitfaden

B.1 Tastenkombinationen

Tastaturen haben ein paar besondere Tasten, die es einem erlauben, zum Beispiel Text zu markieren, kopieren und einzufügen. Auch lassen sich dadurch einige spezielle Zeichen, die oft bei der Programmierung verwendet werden, die auf einer deutschen Tastatur keine eigenen Tasten belegen, eingeben. Die Tasten, die ihr dafür benötigt, sind:

Shift Die sogenannte "Umschalttaste". [1]Sie schaltet zwischen Klein- und Großschreibung, sowie zwischen den verschiedenen abgebildeten Symbolen um.

Feststelltaste Im Englischen "Caps lock", sie befindet oberhalb der "Shift"-Taste, sie ist dafür gedacht, dauerhaft zwisch Groß- und Kleinbuchstaben umzuschalten.

Strg Die Steuerungstaste (im Englischen "Control" bzw. abgekürzt "Ctrl").

Alt "Alt" steht für "Alternative". Sie wird häufig auch in Verbindung mit "Shift" und "Strg" verwendet. Allein wird sie z.B. verwendet, um zwischen Fenstern mit "Alt+Tab" ("Tab" siehe weiter unten) hin- und herzuschalten.

AltGr "AltGr" steht für "Alternative Grafik", ist im Prinzip die "rechte Alt-Taste", hat aber auch die Funktion, Zeichen einzugeben, die auf, z.B. einer deutschen Tastatur, sonst nicht zur Verfügung stehen.

Esc Das ist die sogenannte "escape"-Taste, auf Deutsch übersetzt "Flucht". Mit dieser Taste lassen sich viele Vorgänge abbrechen.

Cursor Damit sind die Tasten gemeint, mit den man nach rechts, links, oben unten geht. Im Deutschen werden sie auch "Kursor", "Schreibmarke", "Eingabezeiger" usw. bezeichnet, oder einfach als "Pfeiltasten".

Tab Die "Tabulator"-Taste dient dazu, Text richtig einzurücken.

[1] Sucht mal auf `https://youtube.com` nach «Umschalttaste mechanische Schreibmaschine», wenn ihr wissen wollt, woher dieser Begriff kommt.

B.1.1 Sonderzeichen

Die folgenden Sonderzeichen gelten für eine deutsche Tastatur:

Tastenkombination	Zeichen	Bedeutung
AltGr + Q	@	At-Zeichen
AltGr + 7	{	Geschweifte Klammer auf
AltGr + 8	[	Eckige Klammer auf
AltGr + 9	]	Eckige Klammer zu
AltGr + 0	}	Geschweifte Klammer zu
AltGr + ß	\	Backslash
AltGr + <	|	Pipe
AltGr + 3	#	Hashtag
AltGr + +	~	Tilde

B.1.2 Referenz

Die folgende Liste von Tastenkombination gilt für viele, aber nicht für alle Programme:

Tastenkombination	Bedeutung
Markieren	
Esc	Markierung aufheben
Shift+Cursor	Text markieren (alternativ mit Maus)
Strg+A	Alles auswählen (“select all”)
Kopieren und Einfügen	
Strg+C	Markierten Text kopieren
Strg+X	Markierten Text ausschneiden
Strg+V	Text einfügen
Strg+Z	Eingabe rückgängig machen (“undo”)
Häufige Funktionen	
Strg+S	Speichern (immer wieder drücken!)
Strg+F	Suchen (“find”)
Strg+H	Suchen und ersetzen (“replace”)

B.2 C

B.2.1 Grundlagen

B.2.2 Datentypen

Ganzzahlen

C und C++ bieten verschiedene Datentypen für Ganzzahlen an, darunter `int`, `short`, `long` und `long long`. Jeder dieser Typen hat eine unterschiedliche Größe und einen unterschiedlichen Wertebereich. Die Größe hängt von der Architektur des Computers ab, auf dem das Programm ausgeführt wird. Diese sind alle **vorzeichenbehaftet**, was bedeutet, dass sie sowohl positive als auch negative Werte darstellen können. Es gibt auch die vorzeichenlosen Varianten `unsigned int`, `unsigned short`, `unsigned long` und `unsigned long long`, die nur nicht-negative Werte darstellen können (vergleicht das nochmal mit dem Abschnitt 1).

Die Liste der ganzzahligen Datentypen in C und C++ ist wie folgt:

`int`
: Standard-Ganzzahltyp, in der Regel 4 Bytes (32 Bit) groß, Wertebereich: -2.147.483.648 bis 2.147.483.647 (vorzeichenbehaftet) oder 0 bis 4.294.967.295 (vorzeichenlos).

`short`
: Kürzere Ganzzahl, in der Regel 2 Bytes (16 Bit) groß, Wertebereich: -32.768 bis 32.767 (vorzeichenbehaftet) oder 0 bis 65.535 (vorzeichenlos).

`long`
: Längere Ganzzahl, in der Regel 4 Bytes (32 Bit) groß, Wertebereich: -2.147.483.648 bis 2.147.483.647 (vorzeichenbehaftet) oder 0 bis 4.294.967.295 (vorzeichenlos).

`long long`
: Noch längere Ganzzahl, in der Regel 8 Bytes (64 Bit) groß, Wertebereich: -9.223.372.036.854.775.808 bis 9.223.372.036.854.775.807 (vorzeichenbehaftet) oder 0 bis 18.446.744.073.709.551.615 (vorzeichenlos).

Gleitkommazahlen

Gleitkommazahlen, also Zahlen mit Nachkommastellen, werden in C und C++ durch die Datentypen `float`, `double` und `long double` dargestellt. Diese Typen unterscheiden sich in der Genauigkeit und dem Wertebereich. Die Liste der Gleitkomma-Datentypen in C und C++ ist wie folgt:

`float`
: Einfachpräzisions-Gleitkommazahl, in der Regel 4 Bytes (32 Bit) groß, Wertebereich: ca. 1.2E-38 bis 3.4E+38

`double`
: Doppelpräzisions-Gleitkommazahl, in der Regel 8 Bytes (64 Bit) groß, Wertebereich: ca. 2.3E-308 bis 1.7E+308

`long double`
: Erweiterte Präzisions-Gleitkommazahl, in der Regel 10, 12 oder 16 Bytes (80, 96 oder 128 Bit) groß, Wertebereich: abhängig von der Implementierung, aber in der Regel größer als `double`

Hier wird eine eigenartige Darstellung von Gleitkommazahlen verwendet, die als **IEEE 754 Standard** bekannt ist. Diese Darstellung ermöglicht es, eine große Bandbreite von Zahlen darzustellen. Das "E" steht für "Exponent" und zeigt an, dass die Zahl in wissenschaftlicher Notation dargestellt wird. Zum Beispiel bedeutet `1.2E-38` die Zahl $1.2 \cdot 10^{-38}$, also eine sehr kleine Zahl, während `3.4E+38` die Zahl $3.4 \cdot 10^{38}$ ist, also eine sehr große Zahl. Man kann sagen, dass die Zahl nach "E" die Zahl der Stellen angibt, um die das Komma verschoben werden muss, um die Zahl in Standardform zu bringen. Ein negativer Exponent bedeutet, dass das Komma nach links verschoben wird, während ein positiver Exponent bedeutet, dass das Komma nach rechts verschoben wird. Damit wird, z.B. die Zahl `1.23E+3` zu 1230 und die Zahl `4.56E-2` zu 0.0456.

Zeichen und Zeichenketten

Zeichen werden in C und C++ durch den Datentyp `char` dargestellt. Eine Zeichenvariable speichert ein einzelnes Zeichen und belegt in der Regel 1 Byte.[2] Zeichenketten sind Arrays von Zeichen, die durch das Nullzeichen `'\0'` terminiert sind. Sie werden durch den Datentyp `char[]` oder `char*` dargestellt.

[2] Bei den Prozessoren, die heutzutage gebaut werden, kann man sich darauf verlassen, dass ein `char` immer 1 einzelnes Byte belegt. Dies war aber in der Geschichte der Computer nicht immer der Fall. Das genau zu beschreiben ist aber ein sehr kompliziertes Thema und eher geeignet für einen Fortgeschrittenenkurs.

`void`

Der Datentyp `void`, was so viel wie "Leere" oder "Lücke" bedeutet, wird verwendet, um anzugeben, dass eine Funktion keinen Rückgabewert hat. Er kann auch in Zeigern verwendet werden, um einen Zeiger auf einen unbekannten Datentyp zu deklarieren.

Zeiger

Zeiger sind Speicheradressen.

`const`

Eine der schwierigsten Themen in C und C++ ist die Verwendung von `const`.

B.2.3 Strukturen

Um komplexere Daten zu speichern, können wir in C und C++ **Strukturen** verwenden. Eine Struktur ist eine Sammlung von Variablen, die unter einem gemeinsamen Namen zusammengefasst sind. Jede Variable innerhalb einer Struktur wird als **Mitglied** bezeichnet. Strukturen ermöglichen es uns, verschiedene Datentypen in einer einzigen Einheit zu kombinieren. Hier ist ein Beispiel für eine Struktur, die einen Punkt im zweidimensionalen Raum darstellt:

```
struct Point {
    int x; // x-Koordinate
    int y; // y-Koordinate
};
```

Wir können dann eine Variable vom Typ `Point` erstellen und auf ihre Mitglieder zugreifen:

```
struct Point p1; // Erstellen einer Point-Variable
p1.x = 10; // Setzen der x-Koordinate
p1.y = 20; // Setzen der y-Koordinate
printf("Punkt: (%d, %d)\n", p1.x, p1.y); // Ausgabe
```

Alternativ können wir auch die Struktur direkt bei der Deklaration initialisieren:

```
struct Point p2 = {30, 40}; // Erstellen mit Initialisierung
printf("Punkt: (%d, %d)\n", p2.x, p2.y); // Ausgabe
```

B.2.4 Anweisungsblock

Ein Anweisungsblock (englisch: *block*) ist eine Gruppe von Anweisungen, die zusammengefasst und als eine Einheit behandelt werden. In C und C++ wird ein Anweisungsblock durch geschweifte Klammern `{ ... }` definiert. Ein Anweisungsblock kann überall dort verwendet werden, wo eine einzelne Anweisung erwartet wird, z.B. in Kontrollstrukturen wie `if`, `for` oder `while`. Innerhalb eines Anweisungsblocks können Variablen deklariert werden, die nur innerhalb dieses Blocks sichtbar sind (lokale Variablen). Dies ermöglicht eine bessere Organisation des Codes und verhindert Namenskonflikte.

Blöcke können auch verschachtelt werden, d.h. ein Anweisungsblock kann innerhalb eines anderen Anweisungsblocks enthalten sein. Dies ist besonders nützlich, um den Gültigkeitsbereich von Variablen zu begrenzen und die Lesbarkeit des Codes zu verbessern.

Aufpassen solltet ihr jedoch mit der Benennung von Variablen in verschachtelten Blöcken. Wenn ihr in einem inneren Block eine Variable mit dem gleichen Namen wie in einem äußeren Block deklariert, wird

die äußere Variable innerhalb des inneren Blocks ”verdeckt”(englisch: *shadowed*). Das bedeutet, dass ihr innerhalb des inneren Blocks nur auf die innere Variable zugreifen könnt, während die äußere Variable vorübergehend nicht zugänglich ist. Dies kann zu Verwirrung führen und sollte vermieden werden, indem ihr eindeutige Namen für Variablen verwendet.Beispiel:

```c
int main() {
    int x = 10; // äußere Variable
    {
        int x = 20; // innere Variable verdeckt die Äußere
        printf("Innere x: %d\n", x); // Ausgabe: 20
    }
    printf("Äußere x: %d\n", x); // Ausgabe: 10
    return 0;
}
```

Es sollte angemerkt werden, dass moderne C- und C++-Compiler in der Regel Warnungen ausgeben, wenn Variablen in verschachtelten Blöcken verdeckt werden. Es ist eine gute Praxis, eindeutige Namen für Variablen zu verwenden, um Verwirrung zu vermeiden.

B.2.5 Kontrollstrukturen

Sowohl in C als auch in C++ gibt es verschiedene Kontrollstrukturen, die den Ablauf eines Programms steuern. Dazu gehören bedingte Anweisungen wie `if`, `else if` und `else`, sowie Schleifen im folgenden Abschnitt. Diese Strukturen ermöglichen es, Entscheidungen zu treffen und bestimmte Codeabschnitte wiederholt auszuführen, basierend auf Bedingungen oder Zählvariablen.

`if`

Die `if`-Anweisung wird verwendet, um eine Bedingung zu überprüfen und einen Codeblock auszuführen, wenn die Bedingung wahr ist. Die Syntax lautet:

```c
if (Bedingung) {
    // Code, der ausgeführt wird, wenn die Bedingung wahr ist
}
```

`else if`

Die `else if`-Anweisung wird verwendet, um eine weitere Bedingung zu überprüfen, wenn die vorherige `if`-Bedingung falsch war. Die Syntax lautet:

```c
if (Bedingung1) {
    // Code, der ausgeführt wird, wenn Bedingung1 wahr ist
} else if (Bedingung2) {
```

```
    // Code, der ausgeführt wird, wenn Bedingung2 wahr ist
}
```

else
: Die else-Anweisung wird verwendet, um einen Codeblock auszuführen, wenn keine der vorherigen Bedingungen wahr ist. Die Syntax lautet:

```
if (Bedingung1) {
    // Code, der ausgeführt wird, wenn Bedingung1 wahr ist
} else if (Bedingung2) {
    // Code, der ausgeführt wird, wenn Bedingung2 wahr ist
} else {
    // Code, der ausgeführt wird, wenn keine der Bedingungen wahr ist
}
```

Eine **Bedingung** ist ein Ausdruck, der entweder `true` (wahr) oder `false` (falsch) ergibt. In C und C++ werden Bedingungen durch Vergleiche oder sogenannte logische Ausdrücke gebildet.

Ein Vergleich bildet sich aus zwei Werten und einem Vergleichsoperator. Wenn wir zum Beispiel feststellen wollen ob zwei Zahlen `a` und `b` gleich sind, dann schreiben wir `a == b`. Dabei wird `==` als Operator bezeichnet (um genau zu sein, als Vergleichsoperator).

Liste aller Vergleichsoperatoren: Mögliche Vergleiche sind:

`==`
: Gleichheit: Überprüft, ob zwei Werte gleich sind.

`!=`
: Ungleichheit: Überprüft, ob zwei Werte ungleich sind.

`<`
: Kleiner als: Überprüft, ob der linke Wert kleiner als der rechte Wert ist.

`>`
: Größer als: Überprüft, ob der linke Wert größer als der rechte Wert ist.

`<=`
: Kleiner oder gleich: Überprüft, ob der linke Wert kleiner oder gleich dem rechten Wert ist.

`>=`
: Größer oder gleich: Überprüft, ob der linke Wert größer oder gleich dem rechten Wert ist.

Logische Operatoren werden verwendet, um mehrere Bedingungen zu kombinieren oder zu negieren. Die wichtigsten logischen Operatoren in C und C++ sind:

&&
: Logisches UND: Gibt `true` zurück, wenn beide Bedingungen wahr sind.

||
: Logisches ODER: Gibt `true` zurück, wenn mindestens eine der Bedingungen wahr ist.

!
: Logisches NICHT: Negiert die Bedingung, d.h. gibt `true` zurück, wenn die Bedingung falsch ist, und `false` zurück, wenn die Bedingung wahr ist.

B.2.6 Schleifen

C und C++ bieten drei Arten von Schleifen an: `for`, `while` und `do-while`. Jede Schleife hat ihre eigenen Anwendungsfälle und Syntax.

`for`-Schleife
: Die `for`-Schleife wird verwendet, wenn die Anzahl der Schritte im Voraus bekannt ist. Sie hat die folgende Syntax:

```
for(Initialisierung; Bedingung; Inkrement) {
    // Code, der wiederholt ausgeführt wird
}
```

`while`-Schleife
: Die `while`-Schleife wird verwendet, wenn die Anzahl der Schritte nicht im Voraus bekannt ist und die Schleife so lange ausgeführt werden soll, wie eine bestimmte Bedingung erfüllt ist. Sie hat die folgende Syntax:

```
while(Bedingung) {
    // Code, der wiederholt ausgeführt wird
}
```

`do-while`-Schleife
: Die `do-while`-Schleife ist ähnlich wie die `while`-Schleife, aber der Code innerhalb der Schleife wird mindestens einmal ausgeführt, bevor die Bedingung überprüft wird. Sie hat die folgende Syntax:

```
do {
    // Code, der wiederholt ausgeführt wird
} while(Bedingung);
```

Die `for`-Schleife nimmt hier eine besondere Stellung ein. Am besten erklärt man die `for`-Schleife, indem man sie in eine `while`-Schleife umwandelt. Die `for`-Schleife

```
for(Initialisierung; Bedingung; Inkrement) {
    // Code, der wiederholt ausgeführt wird
}
```

ist äquivalent zu:

```
{
    Initialisierung;
    while(Bedingung) {
        // Code, der wiederholt ausgeführt wird
        Inkrement;
    }
}
```

Die **Initialisierung** wird nur einmal zu Beginn der Schleife ausgeführt. Sie dient gewöhnlich dazu, eine Zählvariable zu deklarieren und zu initialisieren. Die **Bedingung** wird vor jedem Durchlauf überprüft, und das **Inkrement** wird am Ende jedes Durchlaufs ausgeführt (Inkrement heißt wörtlich Ërhöhung"). Wenn wir zum Beispiel 1 bis 10 zählen wollen, dann schreiben wir:

```
for(int i = 1; i <= 10; i++) {
    printf("%d\n", i);
}
```

oder von 100 in 10-er Schritten bis 0 zählen wollen, dann schreiben wir:

```
for(int i = 100; i >= 0; i -= 10) {
    printf("%d\n", i);
}
```

B.2.7 Funktionen

Um ein Programm in kleinere, wiederverwendbare Teile zu zerlegen, können wir in C und C++ **Funktionen** verwenden. Eine Funktion ist ein benannter Codeblock, der eine bestimmte Aufgabe ausführt und optional Werte zurückgeben kann. Funktionen ermöglichen es uns, den Code zu organisieren, die Lesbarkeit zu verbessern und die Wiederverwendbarkeit zu erhöhen.

Eine Funktion besteht aus einem Funktionskopf, der den Namen der Funktion, die Parameter und den Rückgabewert definiert, und einem Funktionskörper, der die Anweisungen enthält, die ausgeführt werden, wenn die Funktion aufgerufen wird. Die Syntax für die Definition einer Funktion lautet:

```
Rückgabewert Funktionsname(Parameterliste) {
    // Anweisungen
}
```

Der **Rückgabewert** gibt den Datentyp des Wertes an, den die Funktion zurückgibt (siehe oben B.2.2). Wenn die Funktion keinen Wert zurückgibt, wird der Rückgabewert als `void` angegeben. Die **Parameterliste** enthält die Eingabewerte, die an die Funktion übergeben werden, und kann leer sein, wenn keine Eingaben erforderlich sind. Ein Parameter, häufig auch als Argument bezeichnet, ist eine Variable, die innerhalb der Funktion verwendet wird und deren Wert beim Aufruf der Funktion übergeben wird. Ein Argument setzt sich aus Datentype und Variablendamen zusammen. Die Parameterliste kann mehrere Parameter enthalten, die durch Kommas getrennt sind.

Zum Beispiel definieren wir eine Funktion, die zwei ganze Zahlen addiert und das Ergebnis zurückgibt:

```
int addiere(int a, int b) {
    return a + b; // Rückgabe der Summe
}
```

Wir können diese Funktion dann in unserem Hauptprogramm aufrufen:

```
int main() {
    int ergebnis = addiere(5, 3); // Aufruf der Funktion
    printf("Ergebnis: %d\n", ergebnis); // Ausgabe
    return 0;
}
```

B.2.8 Namen

B.2.9 Referenz

`fflush`

Syntax:

```
#include <stdio.h>

int fflush(FILE *stream);
```

Beschreibung:
FIXME

Rückgabewert:
War der Funktionsaufruf erfolgreich, wird 0 zurückgegeben, `EOF` jedoch wenn ein Fehler aufgetreten ist.

`printf`

Syntax:

```
#include <stdio.h>

int printf(const char *format, ...);
```

Beschreibung:
`printf()` schreibt formattierten Text in die Standardausgabe (stdout).
Der `format`-String enthält:

- Normale Zeichen (werden direkt ausgegeben)
- Format-Spezifizierungen (beginnen mit `%`)

Die wichtigsten Format-Spezifizierungen:

`%d`	Ganzzahl (dezimal)
`%f`	Gleitkommazahl
`%c`	Zeichen (char)
`%s`	Zeichenkette (String)
`%x`	Hexadezimal (klein)
`%X`	Hexadezimal (groß)
`%p`	Zeiger (Pointer)

Beispiel:

```c
#include <stdio.h>

int main() {
    int zahl = 42;
    printf("Die Zahl ist: %d\n", zahl);
    printf("Adresse: %p\n", (void*)&zahl);
    return 0;
}

```

Rückgabewert:
Die Anzahl der ausgegebenen Zeichen, oder `-1` bei Fehler.

Siehe auch:
`scanf` (libc), `sprintf` (libc), `fprintf` (libc)

`rand,srand`

Syntax:

```
#include <stdlib.h>

int rand();
void srand(unsigned int seed);
```

Beschreibung:
Der Befehl `rand()` generiert eine Zufallszahl von 0 bis zum dem im Makro `RAND_MAX` definierten Wert. Diese Zufahlszahl wird mit einem Algorithmus berechnet und hängt von ihrem Anfangswert ab (das ist die "seed", auf Deutsch "Saat"). Dieser Anfangswert kann mit dem Befehl `srand` geändert werden.

Beispiel:

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    srand(time(NULL));
    printf("Zufallszahl von %d bis %d: %d\n", 0, RAND_MAX, rand());
    return 0;
}

```

Siehe auch:
`time` (libc)

stdin, stdout, stderr

Syntax:

```
#include <stdio.h>

extern FILE *stdin;
extern FILE *stdout;
extern FILE *stderr;
```

Beschreibung:
Unter normalen Bedingungen hat jedes UNIX-Betriebssystem drei Kanäle offen, die es erlauben, Daten mit einem Benutzer auszutauschen, nämlich

- den Eingabekanal (z.B. für Tastatureingabe),
- den Ausgabekanal (z.B. für Textausgabe auf der Konsole) und
- den Fehlerkanal (der nur für die Ausgabe von Fehlertexten bestimmt ist, meistens auch auf der Konsole).

FIXME

time

Syntax:

```
#include <time.h>

time_t time(NULL);
```

Beschreibung:
Das Kommando `time(NULL)` liefert die Zeit in Form von Sekunden seit dem 1.1.1970, Mitternacht (Weltzeit).

`usleep`

Syntax:

```
#define __GNU_SOURCE
#include <unistd.h>

int usleep(useconds_t usec);
```

Beschreibung:
`usleep()` unterbricht die Ausführung des Programms mindestens `usec` Mikrosekunden. Die genau Wartezeit kann jedoch mehr sein, wenn der Computer sehr viel zu tun hat.

Rückgabewert:
War der Funktionsaufruf erfolgreich, wird `0` zurückgegeben, `-1` jedoch wenn ein Fehler aufgetreten ist.

C
Lösungen zu den Übungen

Lösung 1a
Die vollständige Wahrheitstabelle für die Addition von zwei Bits und den Übertrag sieht folgendermaßen aus:

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Wenn Ihr Euch die Wahrheitstabellen für `AND` und `XOR` in Kapitel 1.2 nochmal anschaut, dann könnt Ihr sehen, dass die Summe S mit der `xor`-Operation berechnet werden kann und der Übertrag C mit der `and`-Operation.

Lösung 1b
Vermutlich wird Euch schon einmal jemand gesagt haben, dass man bei der Lösung einer Aufgabe alle Zwischenschritte aufschreiben soll. Wenn der Autor ehrlich ist: Er hat selbst schon Punkte in Klassenarbeiten verloren, weil er die Zwischenschritte nicht aufgeschrieben hat. Das war ärgerlich – und hätte sich mit einem klaren Plan vermeiden lassen. Das ist aber genau der Punkt bei der Lösung der Aufgabe: ein klarer Plan (oder, wie die Informatiker sagen: ein Algorithmus).

In der Aufgabe wurden die ersten Schritte bereits vorgegeben:

```
S = A xor B
C = A and B
```

Das sieht zunächst folgendermaßen aus:

```
A  00100111       39
B  00000110     +  6
   --------
S  00100001       33
C  00000110        6
```

Das macht jetzt natürlich noch keine Sinn, denn wir wissen ja, dass der *Übertrag* nicht an der gleichen Stelle wie die Summe steht, sondern zum nächsten Bit weiter links gehört. Das bedeutet, dass wir den Übertrag um eine Stelle nach links verschieben müssen:

```
A     00100111       39
B     00000110     +  6
      --------
S     00100001       33
C<<1  00001100       12
```

Auf den ersten Blick sieht das jetzt schon besser aus, aber wir wissen ja, dass wir die Summe und den Übertrag noch einmal addieren müssen. Das bedeutet, dass wir die beiden Zahlen S und $C << 1$ wieder addieren müssen. Das können wir wieder mit den gleichen Schritten tun:

```
S = S xor (C<<1)
C = S and (C<<1)
```

Das sieht dann folgendermaßen aus:

```
S     00100001       33
C<<1  00001100       12
      --------
S     00101101       45
C<<1  00000000        0
```

Jetzt ist der Übertrag C gleich 0, das bedeutet, dass wir die Summe berechnet haben. Die Summe von $A + B$ ist also $S = 00101101$ oder dezimal 45.

Ihr fragt Euch jetzt vielleicht, warum jetzt nochmal eine Addition? Im Kopf macht ihr das mit zweistelligen Zahlen ja auch nicht mehr. Aber schauen wir uns den Algorithmus noch einmal an, indem wir 7 und 3 addieren:

```
A     00000111        7
B     00000011      + 3
      --------
S     00000100        4    Schritt 1
C<<1  00000110        6
      --------
S     00000010        2    Schritt 2
C<<1  00001000        8
```

Jetzt könntet Ihr vermutlich denken, dass wir den Schritt der Berechnung von S und C so lange wiederholen müssen, bis der Übertrag gleich 0 ist, aber tatsächlich ist das nicht notwendig, so dass wir nur noch einmal die beiden Schritte ausführen müssen.

```
S = A xor B
C = A and B
```

```
    S = S xor (C<<1)
    C = S and (C<<1)

    S = S xor (C<<1)
```

Bemerkt hier, dass der Übertrag C in der letzten Zeile nicht mehr berechnet wird, da wir ja wissen, dass er 0 ist (eigentlich müsste man das mathematisch beweisen, aber das lassen wir hier weg). Das bedeutet, dass wir die Summe von $A+B$ mit bitweisen Operation in fünf Schritten berechnen können.[1]

[1] In der Praxis ist es allerding so, dass man auf elektronischer Ebene, also mithilfe von Logikgattern, die Addition in einem Schritt berechnet. Wenn Ihr Zeit habt, dann sucht mal im Internet nach dem Begriff **Volladdierer**.

Lösung 2a

Ab der 7. Programmzeile wird

```
1 double n = 2*i + 1
2 n = 1.0/n;
```

berechnet. Die Reihe wechselt ja ständig das Vorzeichen. Wenn wir nach $i = 1000$ Schritten aufhören, dann ist der Fehler ungefähr so groß wie der nächste Term, den wir nicht mehr berechnet haben. Der nächste Term wäre

$$\Delta\pi = 4\frac{1}{2n+1} = \frac{4}{2001} \approx 0,002\,.$$

Genau genommen ist der Fehler etwas kleiner, weil sich die restlichen Terme gegenseitig teilweise aufheben — aber für eine Schätzung reicht das.

Lösung 2b

Das Programm, um 5 Zufallszahlen auszugeben, sieht zum Beispiel folgendermaßen aus:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    for(int i = 1; i <= 5; i++) {
        int zahl = rand();
        printf("Zufallszahl %d: %d\n", i, zahl);
    }

    return 0;
}
```

Auf dem Computer des Autors ist die Ausgabe folgende:

```
Zufallszahl 1: 1804289383
Zufallszahl 2: 846930886
Zufallszahl 3: 1681692777
Zufallszahl 4: 1714636915
```

```
Zufallszahl 5: 1957747793
```

Möglicherweise seht Ihr genau die gleiche Reihenfolge, das kann aber von Betriebssystem und Compiler abhängen. (Ihr könnt nun im Text nach der Aufgabe im Kapitel 2.7 weiter machen, dort findet Ihr die Erklärung für dieses Verhalten.)

Lösung 2c

a) 11

b) 4 m, 5 Zaunpfähle

c) Wie beim vorhergehenden Zaunpfahlproblem ist `max-min` die Distanz zwischen `min` und `max`. Die Anzahl der Zahlen ist aber `max-min+1`. Wenn ihr also `rand() % (max-min+1)` berechnet, braucht man nur noch `min` dazuzuaddieren:

```
int random_number(int min, int max) {
    return min + rand() % (max-min+1);
}
```

Literatur

Wikipedia (2003a). *Fibonacci-Folge*. URL: `https://de.wikipedia.org/wiki/Fibonacci-Folge` (besucht am 24. 07. 2026) (siehe S. 13).

— (2003b). *Hallo-Welt-Programm*. URL: `https://de.wikipedia.org/wiki/Hallo-Welt-Programm` (besucht am 24. 07. 2026) (siehe S. 9).

— (2026). *Computer*. URL: `https://de.wikipedia.org/wiki/ANSI-Escapesequenz` (besucht am 30. 09. 2026) (siehe S. 21).

Index

Binärsystem, 5

C-Funktion
  `fflush`, 57
  `printf`, 58
  `rand`, 59
  `srand`, 59
  `stderr`, 60
  `stdin`, 60
  `stdout`, 60
  `time`, 60
  `usleep`, 61

C-Makro
  `__GNU_SOURCE`, 19

Lizenz, i, 3